

Assignment one-web pentesting

NAME: SIRAJUDIN SEID

CODE: CTC-153-26

SQL Injection



The image shows a web application interface for 'VulnBank'. At the top, there is a logo of a bank building with a dollar sign, followed by the text 'VulnBank' in a large blue font. Below this, the text 'Secure Login' is centered. There are two input fields: the first one contains the SQL injection payload '' OR 1=1 --', and the second one contains four dots '....'. Below the input fields is a blue button with the text 'Login' in white. The entire form is flanked by two vertical blue bars.

Summary

A SQL Injection vulnerability exists in the application's login functionality. User-controlled input supplied through the `username` parameter is incorporated into a database query without sufficient server-side input validation or parameterized query protection.

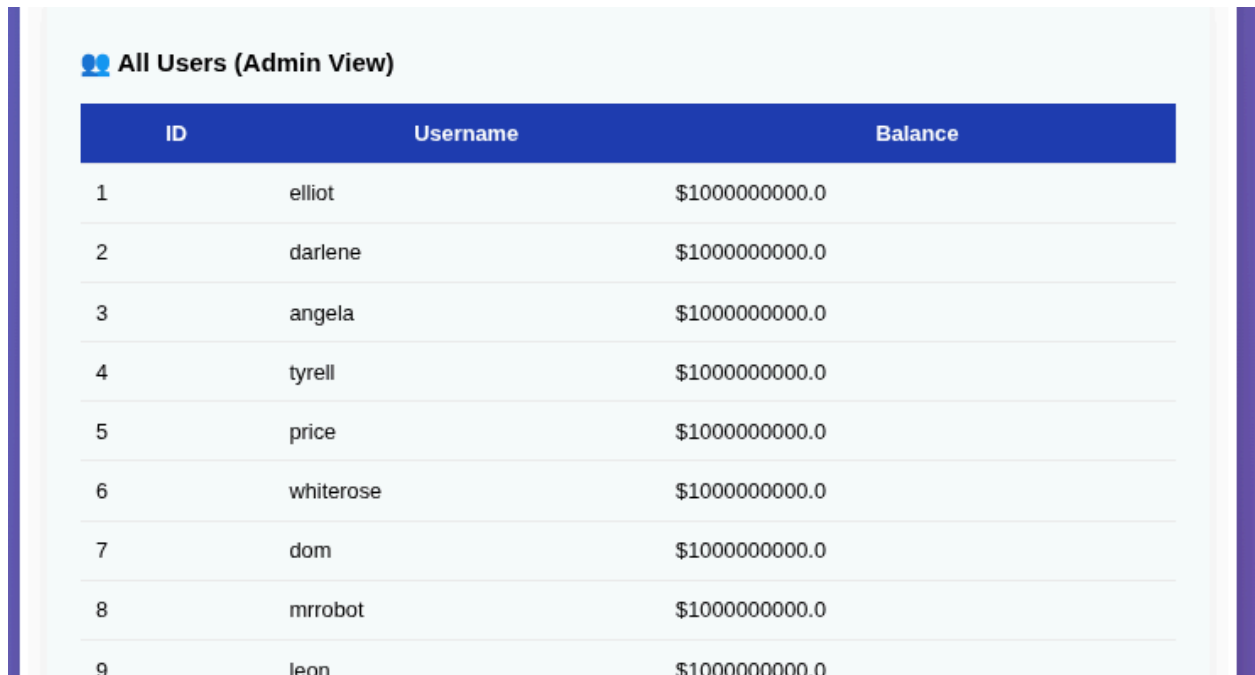
The vulnerability can be demonstrated by supplying a SQL injection payload in the username field:

Reproduction Steps

1. Navigate to the application's login page.
2. Enter the following payload into the Username field:

`' OR 1=1 --`
3. Enter an arbitrary password, for example:

`1234567`
4. Submit the login form.
5. Intercept the request using Burp Suite.
6. Observe that the request contains the attacker-controlled username parameter



ID	Username	Balance
1	elliott	\$10000000000.0
2	darlene	\$10000000000.0
3	angela	\$10000000000.0
4	tyrell	\$10000000000.0
5	price	\$10000000000.0
6	whiterose	\$10000000000.0
7	dom	\$10000000000.0
8	mrrobot	\$10000000000.0
9	leon	\$10000000000.0

Impact

Successful exploitation of this vulnerability could allow an unauthenticated attacker to bypass the application's authentication mechanism

Remediation

The application should never construct SQL queries by concatenating or directly embedding user-controlled input.

Use parameterized queries / prepared statements instead.

Severity :Critical

Race Condition in Money Transfer

Severity: Critical

Summary

A Race Condition vulnerability was identified in the money-transfer functionality.

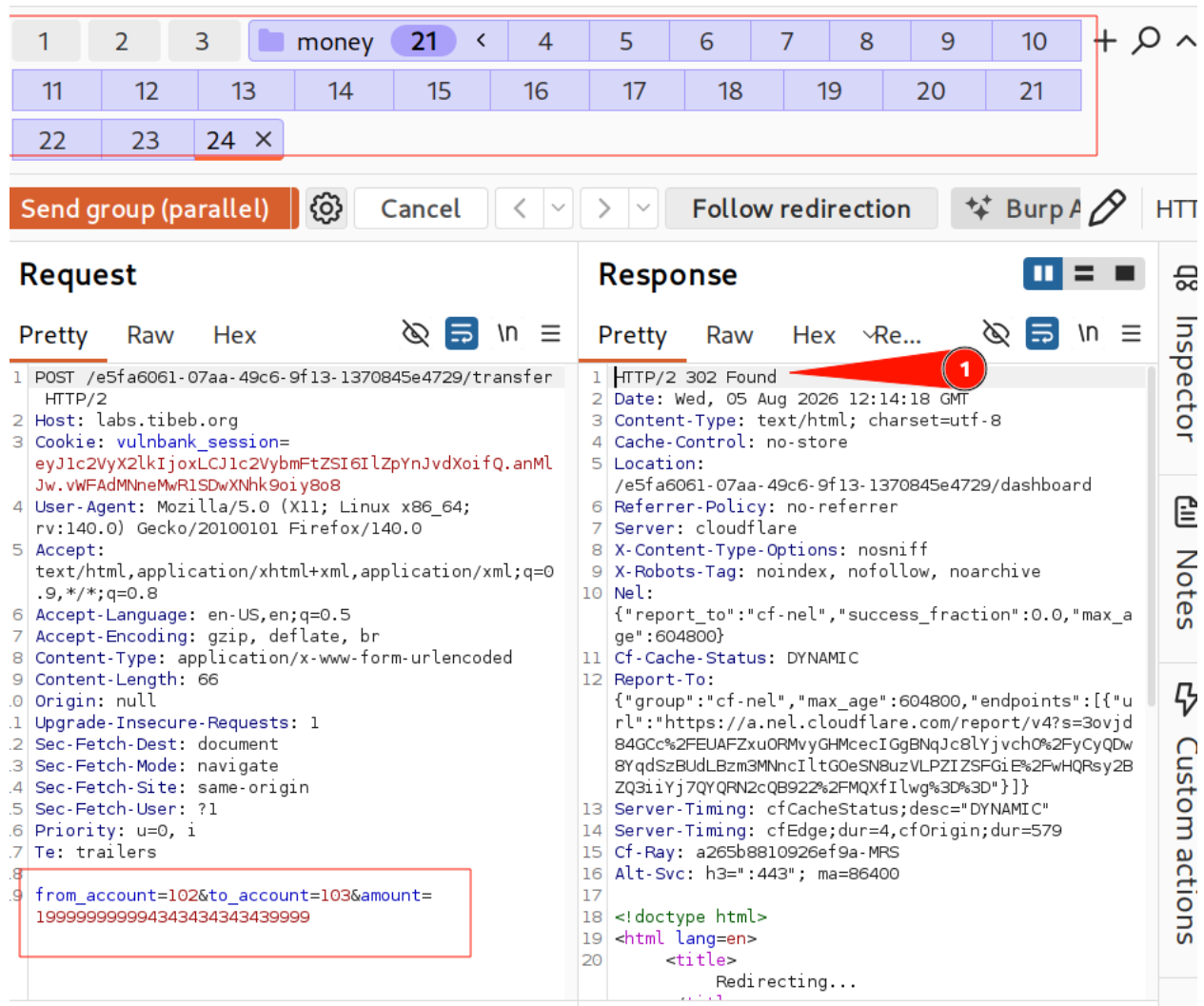
The application appears to process concurrent transfer requests without properly synchronizing access to the account balance. This can allow multiple requests to be processed against the same account state before the balance is correctly updated.

Reproduction Steps

1. Authenticate to the application.
2. Navigate to the money-transfer functionality.
3. Identify the transfer request.
4. Capture the request using Burp Suite.
5. Observe parameters similar to:

`from_account=102to_account=103amount=1999999999943434343434399999`

6. Send multiple transfer requests concurrently rather than sequentially.
7. Observe the application's handling of the simultaneous requests.
8. Verify the balances of the source and destination accounts after the requests complete.
9. If multiple requests are accepted based on the same original account balance, the race condition is confirmed.



Impact

A successful race-condition attack against a financial transaction system can have a critical business impact.

Remediation

The transfer operation should be implemented as an atomic transaction.

The server should ensure that the balance check and balance modification occur as one indivisible database operation

